

Задача 1

$n = 15$, $k = 7$, $d_{\min} = 5$ Поле $\text{GF}(2^4)$ построено по примитивному многочлену $p(x) = x^4 + x + 1$, $\alpha^4 = \alpha + 1$.
Циклотомические классы: $C_1 = \{1, 2, 4, 8\}$, $C_3 = \{3, 6, 12, 9\}$. Минимальные многочлены:

$$m_1(x) = x^4 + x + 1, \quad m_3(x) = x^4 + x^3 + x^2 + x + 1.$$

Порождающий полином

$$g(x) = m_1(x) \cdot m_3(x) = x^8 + x^7 + x^6 + x^4 + 1.$$

1. Систематическая порождающая матрица

Матрица $G = [I_7 \mid P]$, где столбцы P — остатки от деления x^{8+i} на $g(x)$, $i = 0, \dots, 6$.

$$G_{\text{сист}} = \left(\begin{array}{cccccccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{array} \middle| \begin{array}{cccccccc} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{array} \right)$$

2. Декодирование принятой последовательности

Принятый вектор (старший бит слева):

$$\vec{r} = (1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0).$$

Полином $r(x) = x^{14} + x^{13} + x^{12} + x^9 + x^8 + x^4 + x^3$.

2.1. Синдромы

$S_i = r(\alpha^i)$ в $\text{GF}(2^4)$:

$$S_1 = \alpha^{12}, \quad S_2 = S_1^2 = \alpha^9, \quad S_3 = \alpha^{14}, \quad S_4 = S_2^2 = \alpha^3.$$

2.2. Локатор ошибок

$\sigma(x) = 1 + \sigma_1 x + \sigma_2 x^2$. Коэффициенты:

$$\sigma_1 = \alpha^{12}, \quad \sigma_2 = \alpha^{11}.$$

2.3. Позиции ошибок

Корни $\sigma(x)$ в $\text{GF}(2^4)$: α^0 и α^4 . Ошибки на позициях (индексы 14...0): $j = 0$ (младший бит) и $j = 11$.
Исправленное кодовое слово:

$$\vec{c} = (1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1).$$

2.4. Информационные биты

Систематический код: $u = (c_{14} \dots c_8) = (1, 1, 1, 1, 0, 1, 1)$.

0.1 Код на python

```
exp = [0] * 15
log = [0] * 16
val = 1
for i in range(15):
    exp[i] = val
    log[val] = i
    val <<= 1
    if val & 16:
        val ^= 0x13
    val &= 0xF
```

```

def gf_add(a, b): return a ^ b
def gf_mul(a, b):
    if a == 0 or b == 0: return 0
    return exp[(log[a] + log[b]) % 15]
def gf_pow(a, n):
    if a == 0: return 0 if n != 0 else 1
    return exp[(log[a] * n) % 15]

def poly_divmod_int(dividend, divisor):
    quot, rem = 0, dividend
    d_deg = divisor.bit_length() - 1
    while rem.bit_length() > d_deg:
        shift = rem.bit_length() - d_deg - 1
        if (rem >> (d_deg + shift)) & 1:
            rem ^= divisor << shift
            quot ^= 1 << shift
    return quot, rem

def eval_poly_gf16_at(poly_val, point):
    res, v, i = 0, poly_val, 0
    while v:
        if v & 1:
            if point == 0:
                term = 1 if i == 0 else 0
            else:
                term = exp[(log[point] * i) % 15]
            res ^= term
        v >>= 1
        i += 1
    return res

n, k = 15, 7
g = 0b111010001 # x^8 + x^7 + x^6 + x^4 + 1

print(" G = [I7 | P] ")
for i in range(k):
    rem = poly_divmod_int(1 << (8 + i), g)[1]
    info = 1 << (n - 1 - i)
    row = info | rem
    print(f"str {i}: {row:015b} (inf {info:07b} | check {rem:08b})")

print("\nDecode of recieved range")
r_str = "111001100011000"
r_val = int(r_str, 2)
print(f"Принято: {r_str}")

S1 = eval_poly_gf16_at(r_val, exp[1])
S2 = gf_mul(S1, S1)
S3 = eval_poly_gf16_at(r_val, exp[3])
S4 = gf_mul(S2, S2)
print(f"S1 = ^{log[S1]}, S2 = ^{log[S2]}, S3 = ^{log[S3]}, S4 = ^{log[S4]}")

Delta = gf_add(gf_mul(S1, S3), gf_mul(S2, S2))
if Delta:
    invD = exp[(15 - log[Delta]) % 15]
    sigma1 = gf_mul(gf_add(gf_mul(S1, S4), gf_mul(S2, S3)), invD)
    sigma2 = gf_mul(gf_add(gf_mul(S3, S3), gf_mul(S2, S4)), invD)
    print(f"1 = ^{log[sigma1]}, 2 = ^{log[sigma2]}")

```

```

errors = []
for i in range(15):
    val = gf_add(gf_add(1, gf_mul(sigma1, exp[i])), gf_mul(sigma2, exp[(2*i)%15]))
    if val == 0:
        j = (15 - i) % 15
        errors.append(j)
print(f"Errors pos: {sorted(errors)}")

c_val = r_val
for p in errors:
    c_val ^= (1 << p)
print(f"Corrected: {c_val:015b}")
u = (c_val >> 8) & 0x7F
print(f"Inf bits: {u:07b}")

```